```
      where('created_at > ?', 1.week.ago).limit(limit)
5:   end
```

You can also use display to start watching variables. This is a good way of tracking the values of a variable while the execution goes on.

The variables inside the displayed list will be printed with their values after you move in the stack. To stop displaying a variable use undisplay n where n is the variable number (1 in the last example).

3.6 Step by Step

Now you should know where you are in the running trace and be able to print the available variables. But let's continue and move on with the application execution.

Use step (abbreviated s) to continue running your program until the next logical stopping point and return control to the debugger. next is similar to step, but while step stops at the next line of code executed, doing just a single step, next moves to the next line without descending inside methods.

For example, consider the following situation:

```
(byebug)
```

```
(byebug) next
[4, 13] in /PathToProject/app/controllers/articles_controller.rb
```

```
# GET /articles
# GET /articles.json
def index
  @articles = Article.find_recent
  respond_to do |format|
    format.html # index.html.erb
    format.json { render json: @articles }
```